Portfolio Milestone Module 3: Results Summary

Alexander Ricciardi
Colorado State University Global
CSC507: Foundations of Operating Systems
Professor: Dr. Joseph Issa
May 30, 2026

Large output files are common in modern computing because many real systems record activity continuously rather than occasionally. Financial platforms, logistics networks, health systems, and government databases can all produce files with millions of rows in a short period of time. Once files reach that scale, performance becomes an operating-systems issue as much as a programming issue. The operating system must coordinate CPU scheduling, memory use, buffering, storage access, and concurrent work so that the file can be generated or processed efficiently.

In this project, two output files were created with a one-million-line workload. The Bash baseline created `file1.txt` with a shell `for` loop and `$RANDOM`, while the Python benchmark created `file2.txt` and compared four methods: sequential writing, buffered chunk writing, threaded chunk generation, and multiprocessing chunk generation. The final benchmark evidence was captured on Ubuntu 24.04.4 LTS on the `ai01` machine. The system evidence shows an x86_64 platform with 32 logical CPUs, a 13th Gen Intel Core i9-13900KS processor, 125 GiB of RAM, and NVMe solid-state storage. The command-output evidence shows the measured runtimes and the final `wc -l` verification of both generated files.

It is important to note that the same file-generation task can behave very differently depending on how it is implemented. A shell loop, a single Python process, buffered output, threads, and separate worker processes all place different demands on the operating system and runtime. The benchmark results in this project support that point clearly. In particular, the results show that reducing write overhead helps somewhat, but process-based parallelism helps much more when the workload can be divided into independent chunks.

Large Data Files in Real Systems

One common example of a large data file is a financial transaction or audit log. A bank or payment platform may store timestamps, account identifiers, transaction amounts, merchant codes, approval decisions, fraud signals, and audit events. The one-million-row level can be reached quickly because one customer action may create several related records. Processing speed matters because fraud detection, reconciliation, and reporting all depend on current information. If these operations are delayed, the effect can spread across risk analysis, customer service, and compliance work.

A second example is a logistics or package-tracking file. A national shipping system may store label creation, sorting events, truck loading, customs clearance, routing changes, delivery attempts, and final delivery confirmation. In practice, each package can create multiple records, so very large files are expected rather than unusual. Speed matters because routing systems, warehouse dashboards, support tools, and customer-facing tracking pages all depend on timely reads and updates.

A third example is a government record system. A DMV, tax office, or public-benefits system may store identity data, status changes, application records, payment records, and audit events. These files can become very large before corrections, renewals, and case updates are even considered. Processing speed matters because delays can reduce throughput, slow public services, and postpone administrative decisions. In other words, large-file performance affects both technical efficiency and institutional effectiveness.

BENCHMARK DESIGN

The Bash baseline was designed to satisfy the assignment requirement to display the system time before and after the run, delete the prior output file, generate one million random-number lines, and report elapsed time with the Bash `SECONDS` variable (GNU Project, n.d.). This method is simple and correct, but it performs the workload through repeated shell-level loop and append operations.

The Python benchmark was designed to test multiple implementation strategies on the same one-million-line workload. The program measures elapsed time with `time.perf_counter()`, verifies the final line count, and records the results in `benchmark_results.csv`. The benchmark compared four Python methods:

- `sequential`: writes one random number per line in a direct loop
- `buffered`: builds larger chunks and writes them serially
- `threaded`: generates chunks with `ThreadPoolExecutor` and writes them in deterministic order
- `multiprocessing`: generates chunks with `ProcessPoolExecutor` and writes them in deterministic order

The final Ubuntu benchmark used a line count of `1,000,000`, a chunk size of `50,000`, and `32` workers for the parallel methods. That common workload makes it possible to compare the effect of buffering, threading, and multiprocessing without changing the size of the task itself.

COMPARATIVE RUNTIME ANALYSIS

Table 1
Measured Runtime Results for the Module 3 File-Generation Methods

Method	Output file	Runtime
Bash `for` loop with `$RANDOM`	`file1.txt`	`5.000000` seconds
Python sequential loop	`file2.txt`	`0.291220` seconds
Python buffered chunks	`file2.txt`	`0.282278` seconds
Python threaded chunks	`file2.txt`	`0.374363` seconds
Python multiprocessing chunks	`file2.txt`	`0.052204` seconds

Note: The Bash timing and the Python benchmark timings were captured from the Ubuntu `ai01` run shown in `output_terminal.png` and `benchmark_results.csv`.

Table 2
Relative Performance Comparisons

Comparison	Result	Interpretation
Sequential Python vs. Bash	`17.17`x faster	Staying inside one Python process removed much of the shell-loop overhead
Buffered Python vs. sequential Python	`3.1`% faster	Larger writes helped, but only modestly
Threaded Python vs. sequential Python	`28.5`% slower	Thread coordination cost outweighed the benefit for this workload
Multiprocessing Python vs. sequential Python	`5.58`x faster	Separate worker processes made parallel chunk generation effective
Multiprocessing Python vs. Bash	`95.78`x faster	The fastest method combined parallel generation with efficient Python file output

The benchmark results show a clear ranking. The Bash baseline was the slowest method at `5.000000` seconds. This result is not surprising because the shell performs repeated loop work and repeated output operations at a higher overhead level than Python. The sequential Python method reduced the runtime to `0.291220` seconds, which shows that implementation choice matters even before concurrency is introduced.

The buffered Python method improved the sequential result slightly, from `0.291220` seconds to `0.282278` seconds. That is a real improvement, but it is also a small one. The result suggests that

reducing write-call frequency helped, but file output was not the only cost. Random-number generation, string construction, and memory movement still remained part of the total runtime.

The threaded method was slower than both sequential and buffered Python, finishing in `0.374363` seconds. This is an important result because it shows that concurrency is not automatically a performance improvement. Threads add coordination overhead, and the Python documentation notes that executor-based concurrency is only beneficial when the workload is a good match for the execution model (Python Software Foundation, n.d.). In this benchmark, thread management did not provide enough benefit to overcome that overhead.

The multiprocessing method was the strongest result. It completed the full one-million-line workload in `0.052204` seconds, which made it about `5.58`x faster than sequential Python and about `95.78`x faster than the Bash baseline. This result shows that the problem could be divided into independent chunks effectively enough for process-based parallelism to outperform the other methods on the Ubuntu machine.

The final line-count verification also confirmed correctness:

```
1000000 file1.txt
1000000 file2.txt
2000000 total
```

This verification matters because performance claims are only meaningful when the generated output is correct.

OPERATING-SYSTEMS INTERPRETATION

The benchmark results in this project are best understood as an operating-systems comparison rather than only a language comparison. Each method created correct output, but each method used CPU time, memory, scheduling, and file output differently. The Bash baseline depended on repeated shell execution behavior. Sequential Python kept the work inside one runtime process. Buffered Python reduced the number of write operations. Threaded Python introduced concurrency inside one interpreter process. Multiprocessing distributed chunk generation across separate worker processes that the operating system could schedule more independently.

The threaded and multiprocessing methods are especially useful to compare because they separate two kinds of concurrency. Threads share one process and have lower startup cost, but they still require coordination and may not help much when the work is dominated by CPU-heavy Python execution. Separate processes cost more to create, but they can make better use of multiple CPU cores when the work can be partitioned cleanly. In this project, chunk generation was independent enough that multiprocessing became the better match for the hardware and the workload.

The results also show that buffering and parallelism solve different problems. Buffering reduces the cost of repeated writes. Parallelism tries to reduce total compute time by distributing independent

work. The benchmark suggests that the dominant gain in this project came from the second strategy rather than the first one.

CPU-Doubling Estimate

If this system had double its current processing power, the runtime would probably improve, but it would not become exactly twice as fast. Amdahl's Law explains why overall speedup is limited by the portion of the workload that remains serial (Amdahl, 1967). In this benchmark, some work can be parallelized well, especially chunk generation in the multiprocessing method. Other work remains serial or partially serial, including program startup, worker coordination, final ordered writing, and line-count verification.

For that reason, the multiprocessing method would likely benefit the most from additional CPU resources, but perfect 2x scaling is not the logical expectation. The sequential and buffered methods would benefit less because they are still fundamentally single-process workflows. The threaded method would also remain limited if coordination costs continue to outweigh the value of additional concurrency. A reasonable conclusion is that more CPU power would help the multiprocessing case noticeably, but the full benchmark would still be constrained by non-parallel sections of the workflow.

Conclusion

The results of this project show that the file-generation methods behaved differently in ways that operating-systems analysis would predict. The Bash baseline was correct but slowest. Sequential Python was dramatically faster. Buffered Python improved the result slightly, which shows that reducing write overhead helped but did not dominate the workload. Threaded Python was slower, which shows that concurrency must match the problem rather than being assumed to help automatically. Multiprocessing was the fastest method because the work could be divided into independent chunks and scheduled across multiple CPU cores effectively.

Taken together, the benchmark reinforces the main lesson of the assignment: large-file processing should be measured rather than assumed. The operating system, the runtime model, and the implementation strategy all influence performance. For this project, process-based parallelism was the most effective design because it matched both the hardware and the structure of the workload.

REFERENCES

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings, 30*, 483-485. https://doi.org/10.1145/1465482.1465560

GNU Project. (n.d.). *Bash variables*. **In** *GNU Bash manual*. Retrieved May 26, 2026, from https://www.gnu.org/software/bash/manual/html_node/Bash-Variables.html

Python Software Foundation. (n.d.). *concurrent.futures - Launching parallel tasks*. **In** *Python 3.12 documentation*. Retrieved May 26, 2026, from https://docs.python.org/3.12/library/concurrent.futures.html